- Inside the ractor, the ractor can block waiting for another ractor to ask for a value. Metaphorically, the ractor is waiting for somebody to knock on the exit door, and will then send a value out for them. The API call is Ractor.yield(obj), and the argument is the value that is sent out.

The pattern here is that the external calls are messages sent to a ractor and the internal calls are class messages sent to Ractor that know that they take place inside a specific ractor. The API is constrained here to allow for some automatic thread safety to happen as values are passed to a ractor using send or from a ractor using yield or take.

The lifecycle of a ractor goes like this:

First, the ractor is created using Ractor.new. The block is immediately started, any arguments passed to new are passed to the block as though they came from a send message—we haven't talked about what that means yet.

The new ractor is *isolated*, a concept that comes from other languages, but is a new thing for Ruby added just for ractors, which means that the code inside the block will not be able to access any variables that are not defined in the block, no globals, no external locals. The only way to have a value be visible to a ractor is via send.

The code block passed to the ractor executes, until one of the following happens:

- The code block hits a Ractor.yield call, in which case it waits for a different ractor to call ractor.take, at which point it passes away the argument to yield and continues operation.
- The code block hits a Ractor.recieve call, in which case it waits to receive another call to send, the arguments to send are the result of the recieve call, and then continues operation.
- The code block ends. The last expression value is available for one other ractor to retrieve using take.

Let's take a look at how ractors might be used to do the same word count example we just did using fibers:

tut_threads/ractor_word_count.rb

```
reader = Ractor.new(name: "reader") do
  File.foreach("testfile") do |line|
    line.scan(/\w+/) do |word|
      Ractor.yield(word.downcase)
    end
  end
  nil
end

counter = Ractor.new(reader, name: "counter") do |source|
  result = Hash.new(0)
  while(word = source.take)
    result[word] += 1
  end
  result
end

counts = counter.take
counts.keys.sort.each { |k| print "#{k}:#{counts[k]} " }
```

produces:

```
and:1 is:3 line:3 on:1 one:1 so:1 this:3 three:1 two:1
```